

Lecture 4

Deep Learning for Computer Vision – Convolutional Neural Networks and Transfer Learning

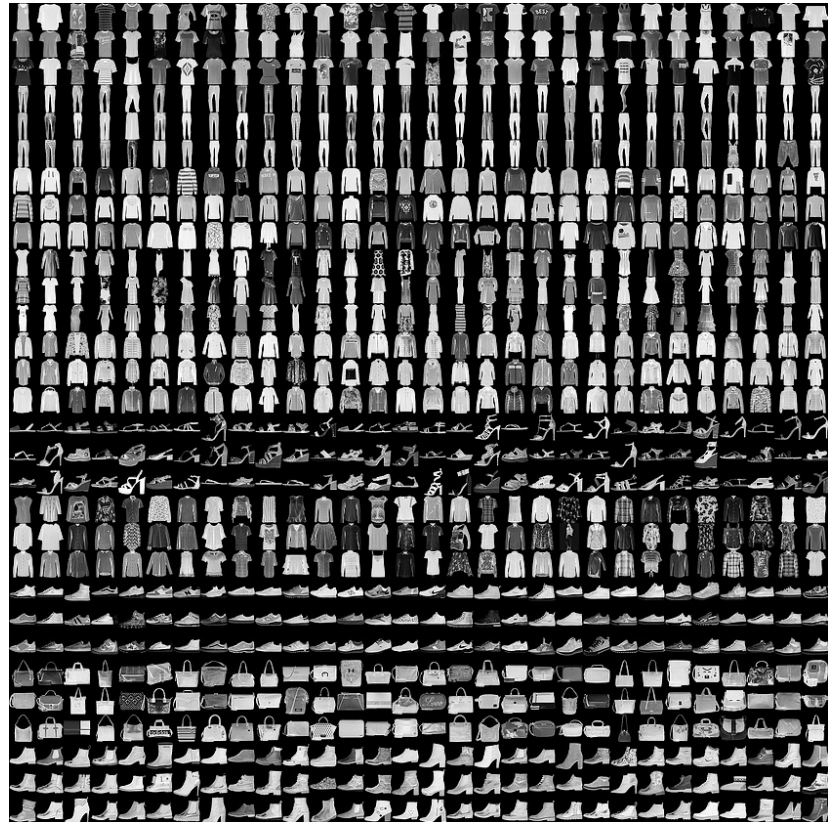


15.S04: Hands-on Deep Learning
Spring 2024
Farias, Ramakrishnan

Fashion MNIST

We saw previously that an NN with a single hidden layer can get to 85% + accuracy on this dataset.

How can we do better?



Sample of the Fashion MNIST images by Yuzamei.
Source: Wikimedia Commons. License: CC BY-SA.

Handling Images “Naturally”

- When we flatten the image matrix into a long vector and feed it to a dense layer, several “undesirable” things happen:

Handling Images “Naturally”

- When we flatten the image matrix into a long vector and feed it to a dense layer, several “undesirable” things happen:
 - We need to learn “too many” parameters
 - Flattening a 3024×3024 – pixel color image (from your phone) and connecting to a single 100-neuron dense layer generates approximately ? parameters.

Handling Images “Naturally”

- When we flatten the image matrix into a long vector and feed it to a dense layer, several “undesirable” things happen:
 - We need to learn “too many” parameters
 - Flattening a 3024×3024 – pixel color image (from your phone) and connecting to a single 100-neuron dense layer generates approximately **2.7 billion** parameters.

Handling Images “Naturally”

- When we flatten the image matrix into a long vector and feed it to a dense layer, several “undesirable” things happen:
 - We need to learn “too many” parameters
 - Flattening a 3024×3024 – pixel color image (from your phone) and connecting to a single 100-neuron dense layer generates approximately **2.7 billion** parameters.
 - This is computationally demanding, very data-hungry and increases the risk of overfitting

Handling Images “Naturally”

- When we flatten the image matrix into a long vector and feed it to a dense layer, several “undesirable” things happen:
 - We need to learn “too many” parameters
 - We lose the *local spatial adjacency* relationships between pixels that define features of the image.

Handling Images “Naturally”

- When we flatten the image matrix into a long vector and feed it to a dense layer, several “undesirable” things happen:
 - We need to learn “too many” parameters
 - We lose the *local spatial adjacency* relationships between pixels that define features of the image.
 - We don’t learn once and reuse repeatedly
 - If a feature of the image (e.g., a vertical line or a circle) appears in different places in the image, the network should “learn it once and use it again and again” rather learn it separately each time.

Handling Images “Naturally”

- When we flatten the image matrix into a long vector and feed it to a dense layer, several “undesirable” things happen:
 - We need to learn “too many” parameters
 - We lose the *local spatial adjacency* relationships between pixels that define features of the image.
 - We don’t learn once and reuse repeatedly
- **Convolutional layers** were developed to address these shortcomings

Convolutional Layers

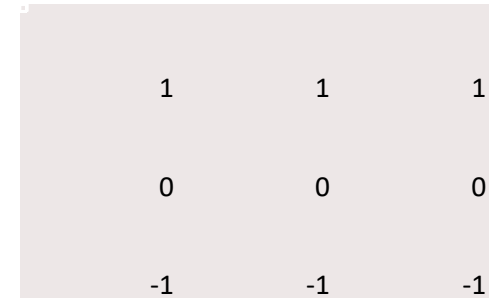
Convolutional Layers and Filters

- *A convolutional filter* is a small square matrix of numbers

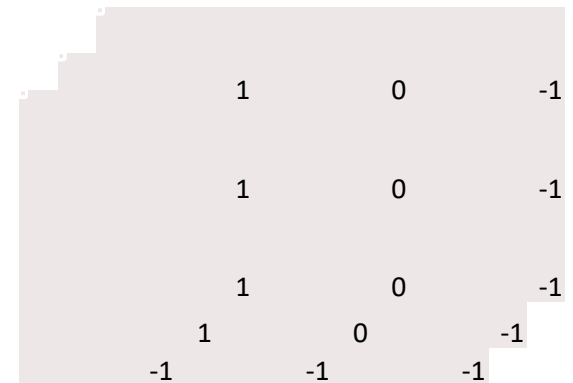
1	1	1
0	0	0
-1	-1	-1

Convolutional Layers and Filters

- *A convolutional filter* is a small square matrix of numbers
- *A convolutional layer* is composed of one or more *convolutional filters*



1	1	1
0	0	0
-1	-1	-1



		1	0	-1
		1	0	-1
		1	0	-1
	1	0	-1	
-1	-1	-1		

The Convolutional Filter

By choosing the numbers in a filter carefully and “applying” the filter to an image, different features of the image can be detected (as we will demonstrate shortly)

The Convolutional Filter

By choosing the numbers in a filter carefully and “applying” the filter to an image, different features of the image can be detected (as we will demonstrate shortly)

This filter can detect horizontal lines!

1	1	1
0	0	0
-1	-1	-1

This filter can detect vertical lines

1	0	-1
1	0	-1
1	0	-1

Applying a convolutional filter to an image is called the “convolution operation”

Input

0	0	0	0	0	0	0
5	5	5	5	5	5	5
10	10	10	10	10	10	10
15	15	15	15	15	15	15
10	10	10	10	10	10	10
5	5	5	5	5	5	5
0	0	0	0	0	0	0

*

Filter

1	1	1
0	0	0
-1	-1	-1

=

Output

Applying a convolutional filter to an image is called the “convolution operation”

Input

0	0	0	0	0	0	0
5	5	5	5	5	5	5
10	10	10	10	10	10	10
15	15	15	15	15	15	15
10	10	10	10	10	10	10
5	5	5	5	5	5	5
0	0	0	0	0	0	0

Filter

1	1	1
0	0	0
-1	-1	-1

*

=

Output

The convolution operation

- “Overlay” the filter onto the top-left of the image

Applying a convolutional filter to an image is called the “convolution operation”

Input

0	0	0	0	0	0	0
5	5	5	5	5	5	5
10	10	10	10	10	10	10
15	15	15	15	15	15	15
10	10	10	10	10	10	10
5	5	5	5	5	5	5
0	0	0	0	0	0	0

Filter

1	1	1
0	0	0
-1	-1	-1

*

=

Output

The convolution operation

- “Overlay” the filter onto the top-left of the image
- Multiply matching elements and add up:
 - $0*1 + 0*1 + 0*1 + 5*0 + 5*0 + 5*0 + 10*-1 + 10*-1 + 10*-1 = -30$

Applying a convolutional filter to an image is called the “convolution operation”

Input

0	0	0	0	0	0	0
5	5	5	5	5	5	5
10	10	10	10	10	10	10
15	15	15	15	15	15	15
10	10	10	10	10	10	10
5	5	5	5	5	5	5
0	0	0	0	0	0	0

Filter

1	1	1
0	0	0
-1	-1	-1

*

=

Output

0						

The convolution operation

- “Overlay” the filter onto the top-left of the image
- Multiply matching elements and add up:
 - $0*1 + 0*1 + 0*1 + 5*0 + 5*0 + 5*0 + 10*-1 + 10*-1 + 10*-1 = -30$
- Run through a ReLU*: $\max(0, -30) = 0$. This number is the top-left cell of the output

*strictly speaking, the ReLU isn't part of the convolution operation but we include it here for convenience

The Convolution Operation

Input

0	0	0	0	0	0	0
5	5	5	5	5	5	5
10	10	10	10	10	10	10
15	15	15	15	15	15	15
10	10	10	10	10	10	10
5	5	5	5	5	5	5
0	0	0	0	0	0	0

Filter

1	1	1
0	0	0
-1	-1	-1

*

=

Output

0	0					

Slide the window one step to the right and repeat this process to get the second number of the output

The Convolution Operation

Input

0	0	0	0	0	0	0
5	5	5	5	5	5	5
10	10	10	10	10	10	10
15	15	15	15	15	15	15
10	10	10	10	10	10	10
5	5	5	5	5	5	5
0	0	0	0	0	0	0

Filter

1	1	1
0	0	0
-1	-1	-1

*

=

Output

0	0	0	0	0

Slide the window one step to the right and repeat this process to get the second number of the output

When done with the first row, move to the start of the second row and continue as before ...

The Convolution Operation

Input

0	0	0	0	0	0	0
5	5	5	5	5	5	5
10	10	10	10	10	10	10
15	15	15	15	15	15	15
10	10	10	10	10	10	10
5	5	5	5	5	5	5
0	0	0	0	0	0	0

Filter

1	1	1
0	0	0
-1	-1	-1

*

=

Output

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
30	30	30	30	30	30	30
30	30	30	30	30	30	30

Slide the window one step to the right and repeat this process to get the second number of the output

When done with the first row, move to the start of the second row and continue as before ...

... till you reach the bottom-right corner

By choosing the numbers in a filter carefully and applying the convolution operation, different features of the image can be detected



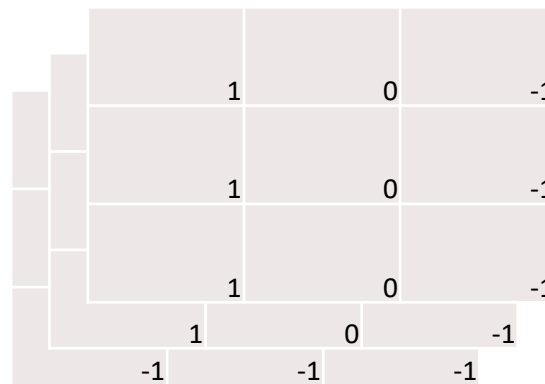
Switch to HODL-Lec-3-Convolution-Example.xlsx

Optional:

Check out <https://setosa.io/ev/image-kernels/> to practice with different filters

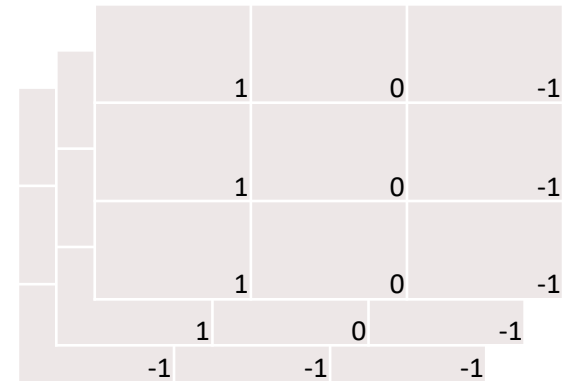
Convolutional Layers

A convolutional layer is composed of one or more convolutional filters



Convolutional Layers

A *convolutional layer* is composed of one or more convolutional filters



Each filter can be thought of as a specialist for detecting a particular feature (e.g., a horizontal line, an arc, a vertical line)

Applying a Convolutional *Layer* to an image

Input image

0	0	0	0	0	0	0
5	5	5	5	5	5	5
10	10	10	10	10	10	10
15	15	15	15	15	15	15
10	10	10	10	10	10	10
5	5	5	5	5	5	5
0	0	0	0	0	0	0

Conv layer
with 2 filters

1	1	1
0	0	0
-1	-1	-1

1	0	-1
1	0	-1
1	0	-1

Output

Applying a Convolutional Layer to an image

Input image

0	0	0	0	0	0	0
5	5	5	5	5	5	5
10	10	10	10	10	10	10
15	15	15	15	15	15	15
10	10	10	10	10	10	10
5	5	5	5	5	5	5
0	0	0	0	0	0	0

Conv layer
with 2 filters

1	1	1
0	0	0
-1	-1	-1

1	0	-1
1	0	-1
1	0	-1

Output

0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
30	30	30	30	30
30	30	30	30	30

Applying a Convolutional Layer to an image

Input image

0	0	0	0	0	0	0
5	5	5	5	5	5	5
10	10	10	10	10	10	10
15	15	15	15	15	15	15
10	10	10	10	10	10	10
5	5	5	5	5	5	5
0	0	0	0	0	0	0

Conv layer
with 2 filters

1	1	1
0	0	0
-1	-1	-1

1	0	-1
1	0	-1
1	0	-1

Output

0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
30	30	30	30	30
30	30	30	30	30

0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

Applying a Convolutional Layer to an image

Input image

0	0	0	0	0	0	0
5	5	5	5	5	5	5
10	10	10	10	10	10	10
15	15	15	15	15	15	15
10	10	10	10	10	10	10
5	5	5	5	5	5	5
0	0	0	0	0	0	0

Conv layer
with 2 filters

1	1	1
0	0	0
-1	-1	-1

1	0	-1
1	0	-1
1	0	-1

Output

0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
30	30	30	30	30
30	30	30	30	30

0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

These two 5x5 matrices can be represented as a tensor of shape _____?

Applying a Convolutional Layer to an image

Input image

0	0	0	0	0	0	0
5	5	5	5	5	5	5
10	10	10	10	10	10	10
15	15	15	15	15	15	15
10	10	10	10	10	10	10
5	5	5	5	5	5	5
0	0	0	0	0	0	0

Conv layer
with 2 filters

1	1	1
0	0	0
-1	-1	-1

1	0	-1
1	0	-1
1	0	-1

Output

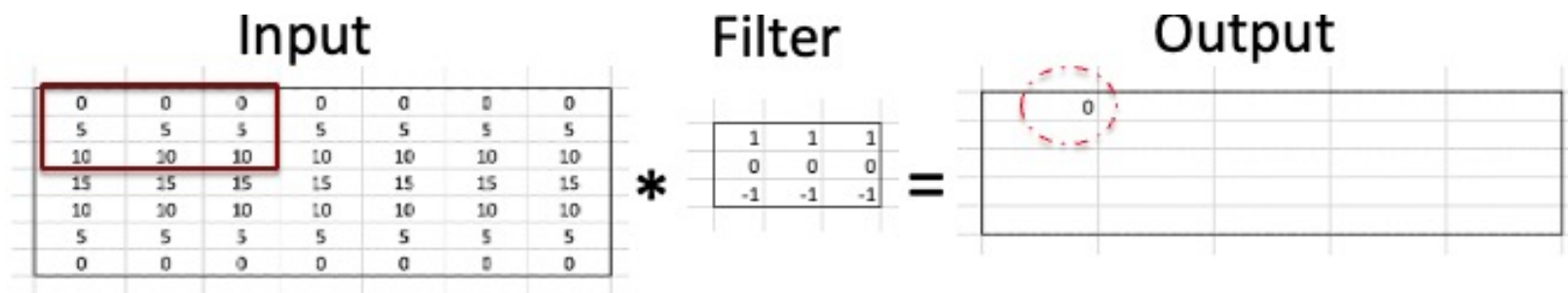
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
30	30	30	30	30
30	30	30	30	30

0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

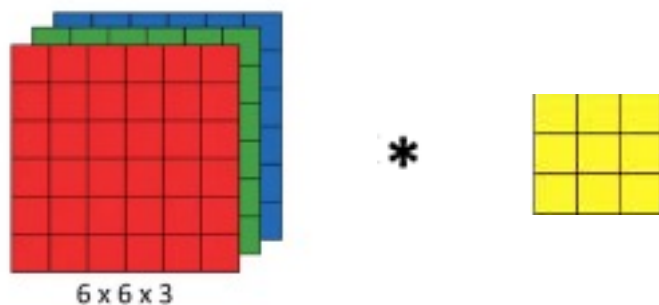
These two 5x5 matrices can be represented as a tensor of shape 5 x 5 x 2 or 2 x 5 x 5

Applying a Convolutional Filter to a color image

We know how to apply a convolutional filter to a 2-d tensor (e.g., a grayscale image)

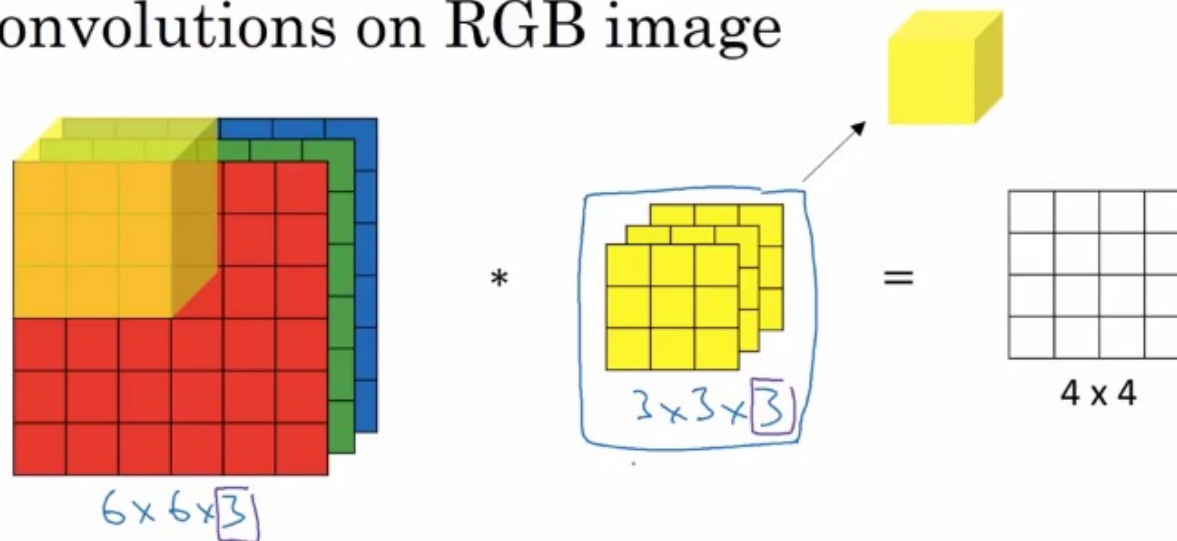


How should we apply a convolutional filter to a rank-3 tensor (e.g., a color image)?



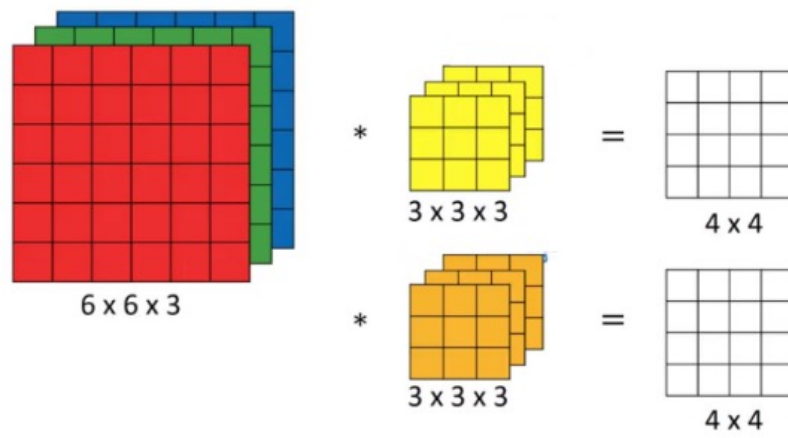
Applying a Convolutional Filter to a color image

Convolutions on RGB image



- We make the filter rank-3 as well and give it the same depth as the input
- The other aspects of the convolution operation are unchanged

Applying a Convolutional Layer to a color image



If we had instead applied 2 filters, the output would be a tensor with shape $4 \times 4 \times 2$

If we had instead applied f filters, the output would be a tensor with shape $4 \times 4 \times f$



Please see Chapter 8.1 of the textbook
for more detail on how convolutional
filters and layers work

The Big Idea

- These filters seem excellent but how are we supposed to come up with the numbers in each filter?

The Big Idea

- These filters seem excellent but how are we supposed to come up with the numbers in each filter?
- In fact, convolutional filters used to be designed by hand. Computer Vision researchers invested a lot of effort in devising filters that could detect various types of image features

The Big Idea

- These filters seem excellent but how are we supposed to come up with the numbers in each filter?
- In fact, convolutional filters used to be designed by hand. Computer Vision researchers invested a lot of effort in devising filters that could detect various types of image features
- As we figured out how to train deep networks with lots of weights, a big idea emerged: **think of the numbers in the filter as weights and simply learn them from the data, just like we learn all the other weights**

The Big Idea

- These filters seem excellent but how are we supposed to come up with the numbers in each filter?
- In fact, convolutional filters used to be designed by hand. Computer Vision researchers invested a lot of effort in devising filters that could detect various types of image features
- As we figured out how to train deep networks with lots of weights, a big idea emerged: **think of the numbers in the filter as weights and simply learn them from the data, just like we learn all the other weights**
 - This is possible because a convolutional filter is just a neuron like we saw in previous lectures, albeit a special one (see appendix)

The Big Idea

- These filters seem excellent but how are we supposed to come up with the numbers in each filter?
- In fact, convolutional filters used to be designed by hand. Computer Vision researchers invested a lot of effort in devising filters that could detect various types of image features
- As we figured out how to train deep networks with lots of weights, a big idea emerged: **think of the numbers in the filter as weights and simply learn them from the data, just like we learn all the other weights**
 - This is possible because a convolutional filter is just a neuron like we saw in previous lectures, albeit a special one (see appendix)
 - Therefore, our entire machinery – neurons, layers, loss functions, gradient descent – is perfectly applicable

The Big Idea

- These filters seem excellent but how are we supposed to come up with the numbers in each filter?
- In fact, convolutional filters used to be designed by hand. Computer Vision researchers invested a lot of effort in devising filters that could detect various types of image features
- As we figured out how to train deep networks with lots of weights, a big idea emerged: **think of the numbers in the filter as weights and simply learn them from the data, just like we learn all the other weights**
- This turned out to be a turning point in the Computer Vision field and led to massive improvements in algorithmic capabilities

Later conv layers “see” more of the original input than the earlier layers

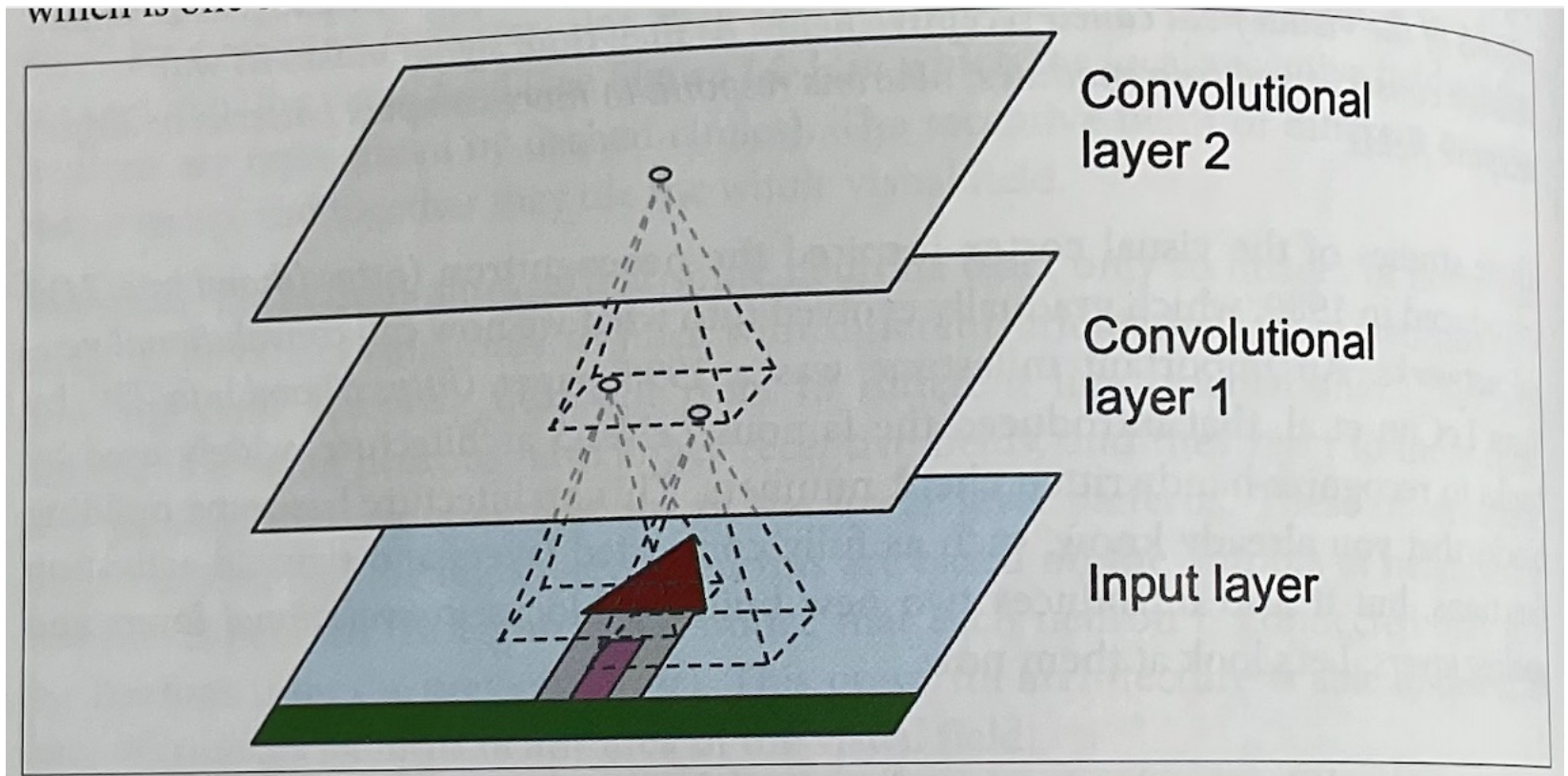
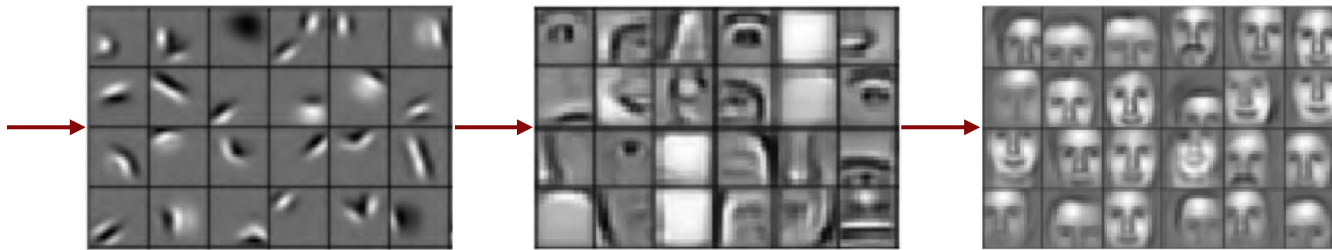


Image credit: Fig 14-2 from <https://www.amazon.com/Hands-Machine-Learning-Scikit-Learn-TensorFlow/dp/1492032646>

As a result, a network with *many* convolutional layers can learn increasingly complex features



lines => edges, circles => faces!

Convolutional layers images © Honglak Lee, Roger Grosse, Rajesh Ranganath, Andrew Y. Ng. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use>.

Convolutional Deep Belief Networks for Scalable Unsupervised Learning of Hierarchical Representations by Lee et al (2009)

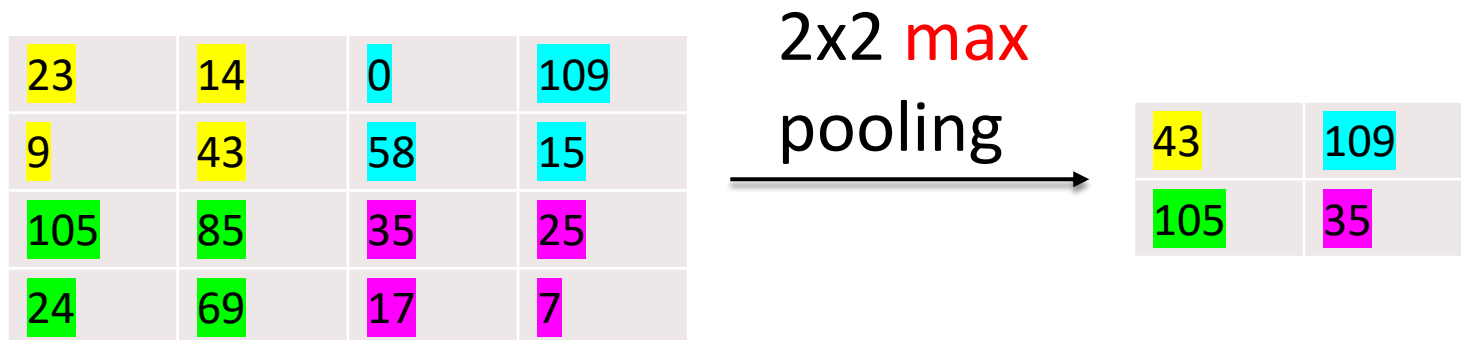
Pooling Layers

Pooling Layers

Pooling layers (also called down-sampling or subsampling layers) reduce the size of the tensor coming out of a convolutional layer

Pooling Layers

Pooling layers (also called down-sampling or subsampling layers) reduce the size of the tensor coming out of a convolutional layer



- In *max-pooling*, we take the **maximum** value from each 2x2 box

Pooling Layers

Pooling layers (also called down-sampling or subsampling layers) reduce the size of the tensor coming out of a convolutional layer

23	14	0	109
9	43	58	15
105	85	35	25
24	69	17	7

2x2 **average**
pooling

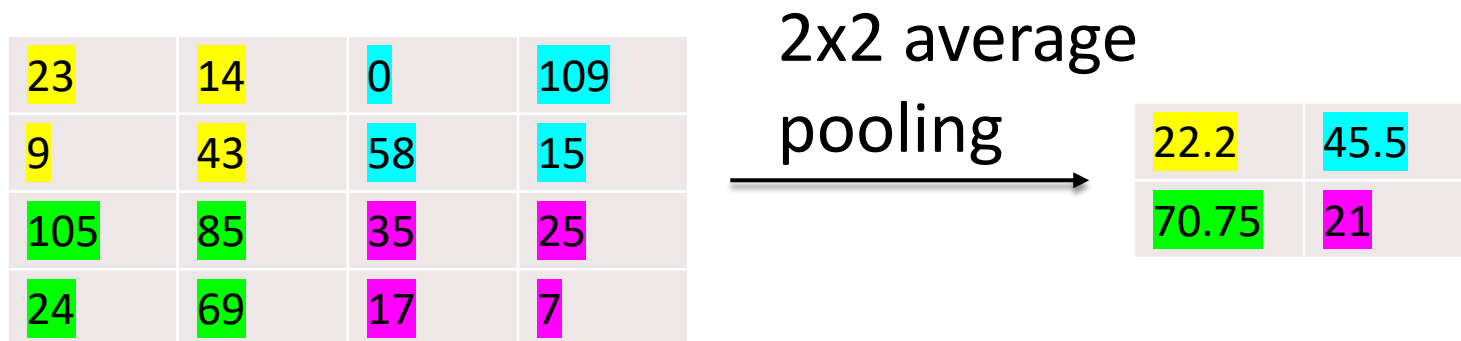


22.2	45.5
70.75	21

- In *max-pooling*, we take the maximum value from each 2x2 box
- In *average pooling*, we take the **average** of each 2x2 box

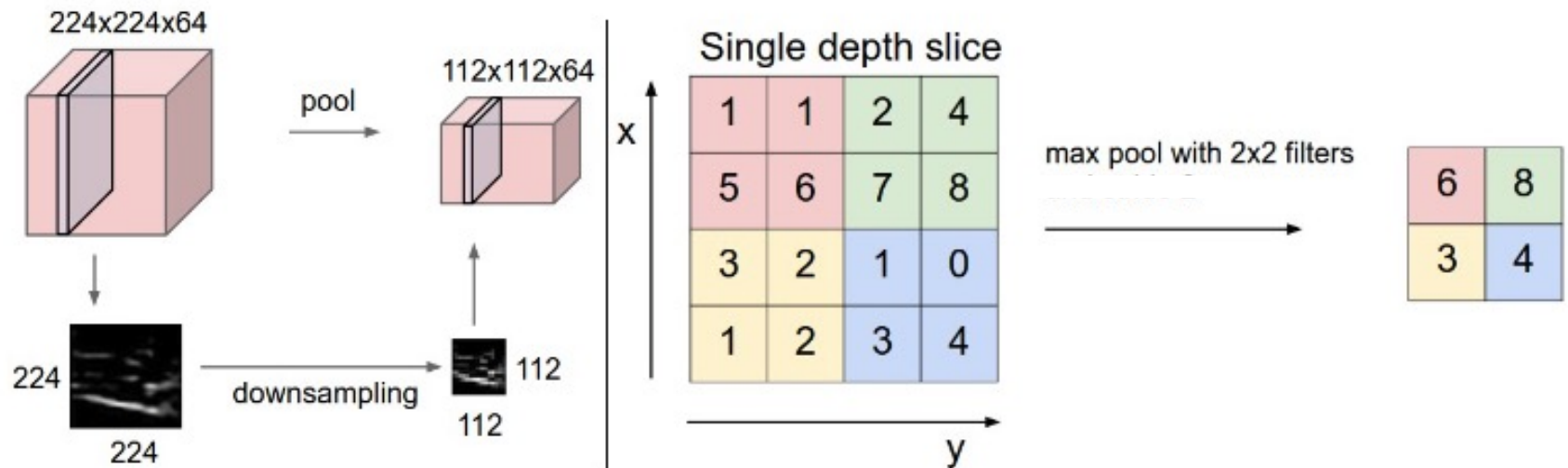
Pooling Layers

Pooling layers (also called down-sampling or subsampling layers) reduce the size of the tensor coming out of a convolutional layer



- Reduces the number of entries significantly (e.g., 75% for 2x2 pooling)
- The output from the pooling layer is fed to the next layer as usual

Pooling Layers



Intuition behind max pooling

- Max pooling acts like an “OR” condition: if a feature exists anywhere in its input, max-pooling will pick it up i.e., max-pooling acts like a *feature detector*

Intuition behind max pooling

- Max pooling acts like an “OR” condition: if a feature exists anywhere in its input, max-pooling will pick it up i.e., max-pooling acts like a *feature detector*

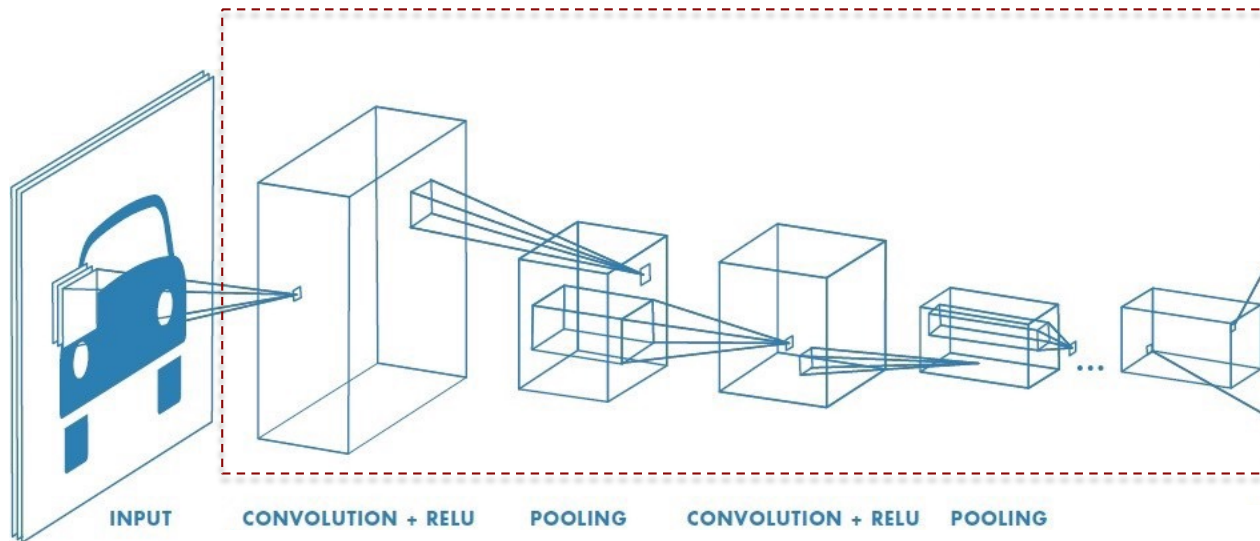
Switch to Notability

Intuition behind max pooling

- Max pooling acts like an “OR” condition: if a feature exists anywhere in its input, max-pooling will pick it up i.e., max-pooling acts like a *feature detector*
- Since successive convolutional layers can “see” more and more of the original input image, the max-pooling layers that follow them can detect if a feature exists in more and more of the original input image as well

The Architecture of a Basic CNN

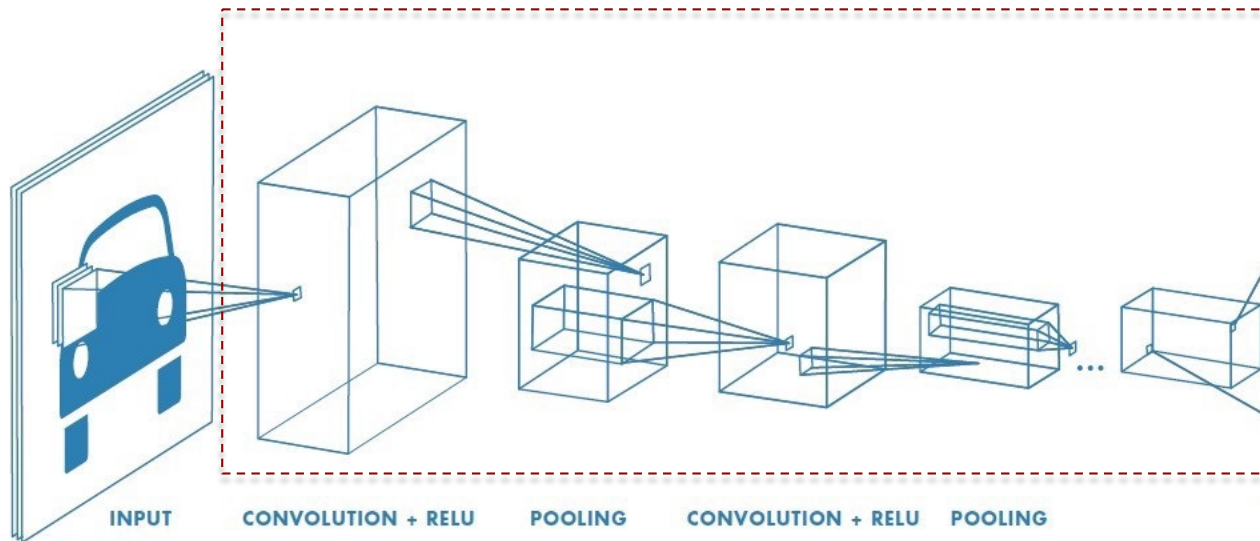
The architecture of a basic CNN



A series of **convolutional blocks**

Convolutional blocks image by Faisal Alshuwaier, Ali Areshey, Josiah Poon. License: CC BY-NC-ND. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use>.

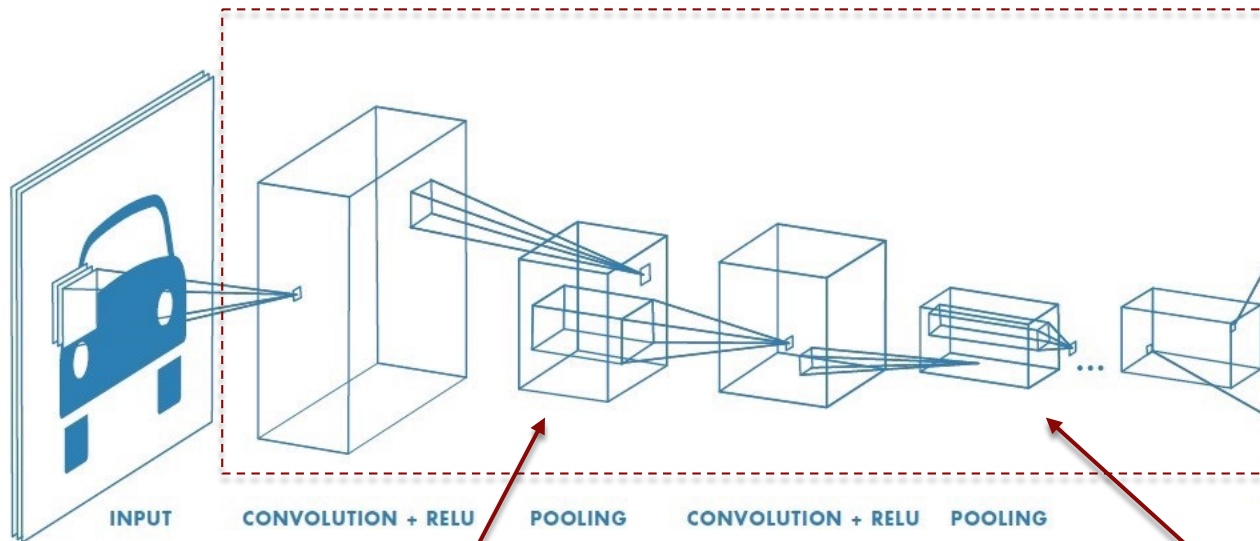
The architecture of a basic CNN



Each *convolutional block* typically has 1-2 convolutional layers followed by a pooling layer

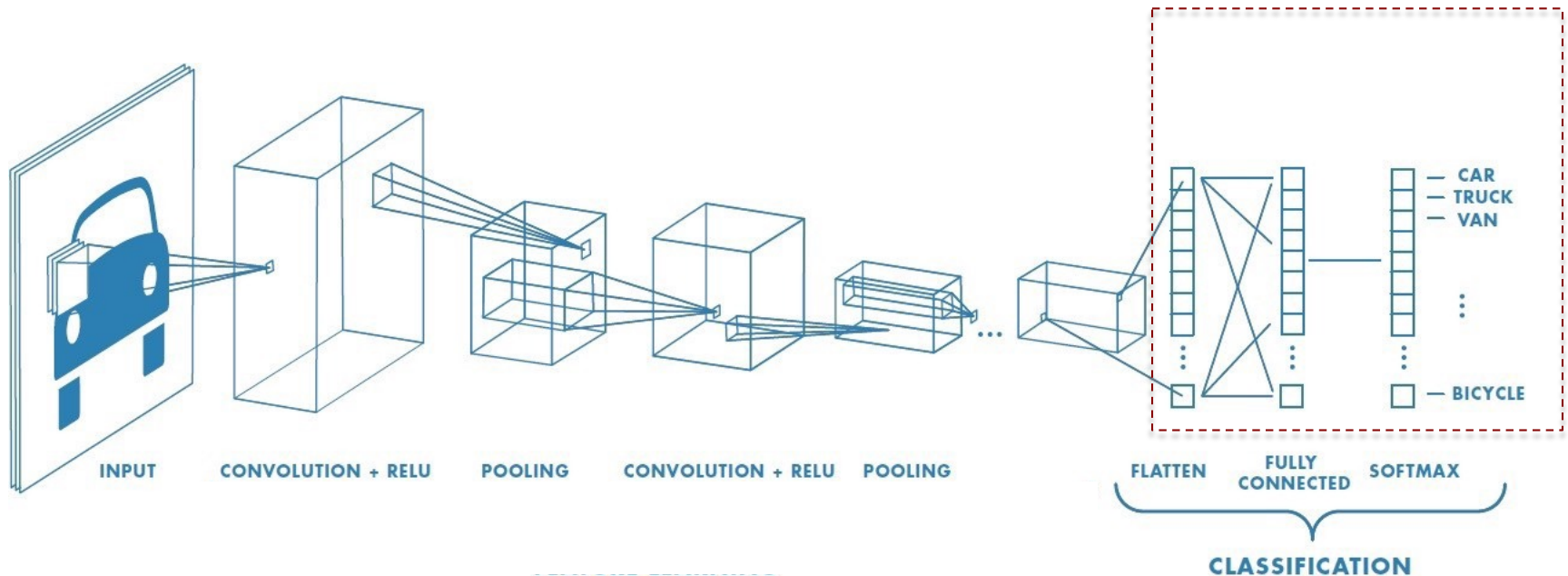
Convolutional blocks image by Faisal Alshuwaier, Ali Areshey, Josiah Poon. License: CC BY-NC-ND. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use>.

The architecture of a basic CNN



Each block will typically have more depth than the previous block but lower height/width. Compare this to this

At the end, we flatten the tensor, run it through fully-connected layers, and then the output layer



Convolutional blocks image by Faisal Alshuwaier, Ali Areshey, Josiah Poon.
License: CC BY-NC-ND. This content is excluded from our Creative Commons
license. For more information, see <https://ocw.mit.edu/help/faq-fair-use>.

The final tensor gets flattened into a long vector and sent through 0 or more hidden layers to the output layer

Colab: Let's solve Fashion MNIST with
Convolutional layers!

[Link to colab](#)

Next: We will work with color images

Motivating application: A Handbags-Shoes Classifier based on less than 100 images!

Web-scraped dataset of < 100 color images of handbags and shoes

With this **tiny dataset**, we will build a deep learning network to classify your shoe or handbag in class with high accuracy!



Generic images of handbags and shoes © unknown. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use>.

Colab: Let's build a shoes-handbags classifier with Convolutional layers!

[Link to colab](#)

Can we do better? We only have
100 examples of each class

Transfer Learning with Pre-trained Networks

Transfer learning takes advantage of two research trends

Trend 1: Researchers have designed NN architectures that are well-matched to different *types* of data. For example:

Type of data	Architecture
All	Residual connections
Images	Convolutional layers
Sequences (e.g., natural language, audio, video, gene sequences)	Transformers

Transfer learning takes advantage of two research trends

Trend 1: Researchers have designed NN architectures that are well-matched to different *types* of data. For example:

Type of data	Architecture
All	Residual connections
Images	Convolutional layers
Sequences (e.g., natural language, audio, video, gene sequences)	Transformers

Trend 2: Using these architectural innovations, researchers have trained high-performance DNNs on a variety of large real-world datasets. **Numerous pretrained models are available!**



Transfer learning involves **customizing** such a **pre-trained network to your problem**, rather than designing and training a network from scratch.

Can we apply Transfer Learning to build a better Handbags/Shoes Classifier?*

Handbags and shoes are “everyday objects” and you can look around and see if there are any networks that have been trained on a dataset of images of “everyday objects”

*The first thing to do, of course, is to see if someone has already built such a classifier and put it on GitHub



Can we apply Transfer Learning to build a better Handbags/Shoes Classifier?

Handbags and shoes are “everyday objects” and you can look around and see if there are any networks that have been trained on a dataset of images of “everyday objects”

Turns out the ImageNet dataset has millions of images of 1000 categories of everyday objects. We can look for networks that have been trained on ImageNet

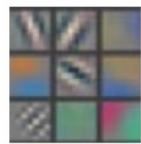
IMAGENET

- 1,000 object classes (categories).
- Images:
 - 1.2 M train
 - 100k test.

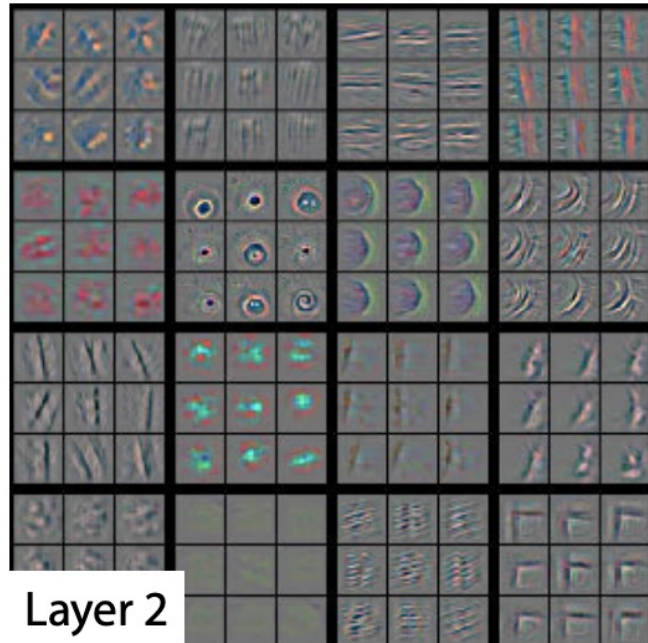
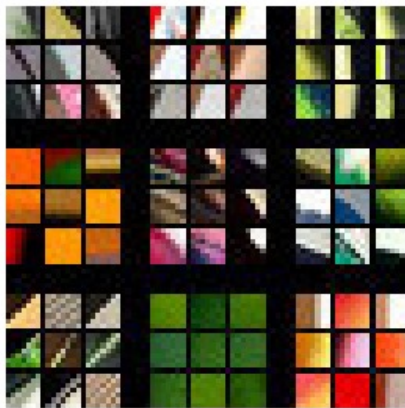


<https://www.slideshare.net/xavigiro/image-classification-on-imagenet-d1l4-2017-upc-deep-learning-for-computer-vision/>

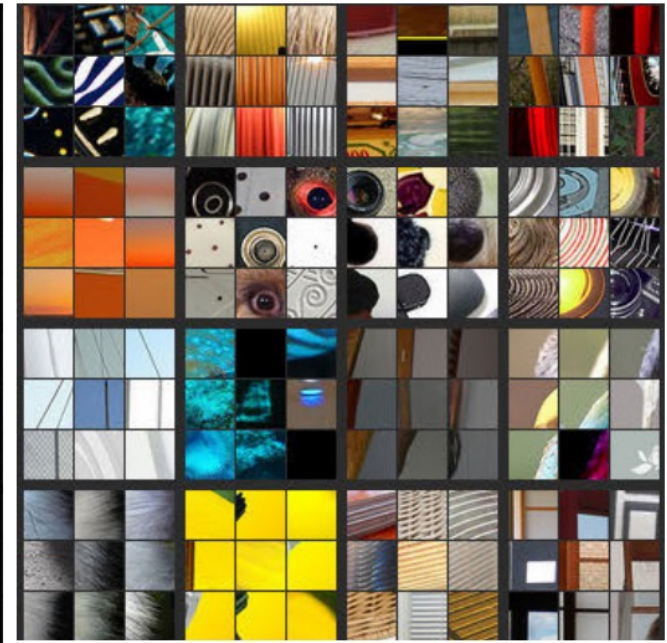
A network trained on ImageNet and that does well on it has essentially developed a **smart, hierarchical representation** of these objects **across its layers**



Layer 1

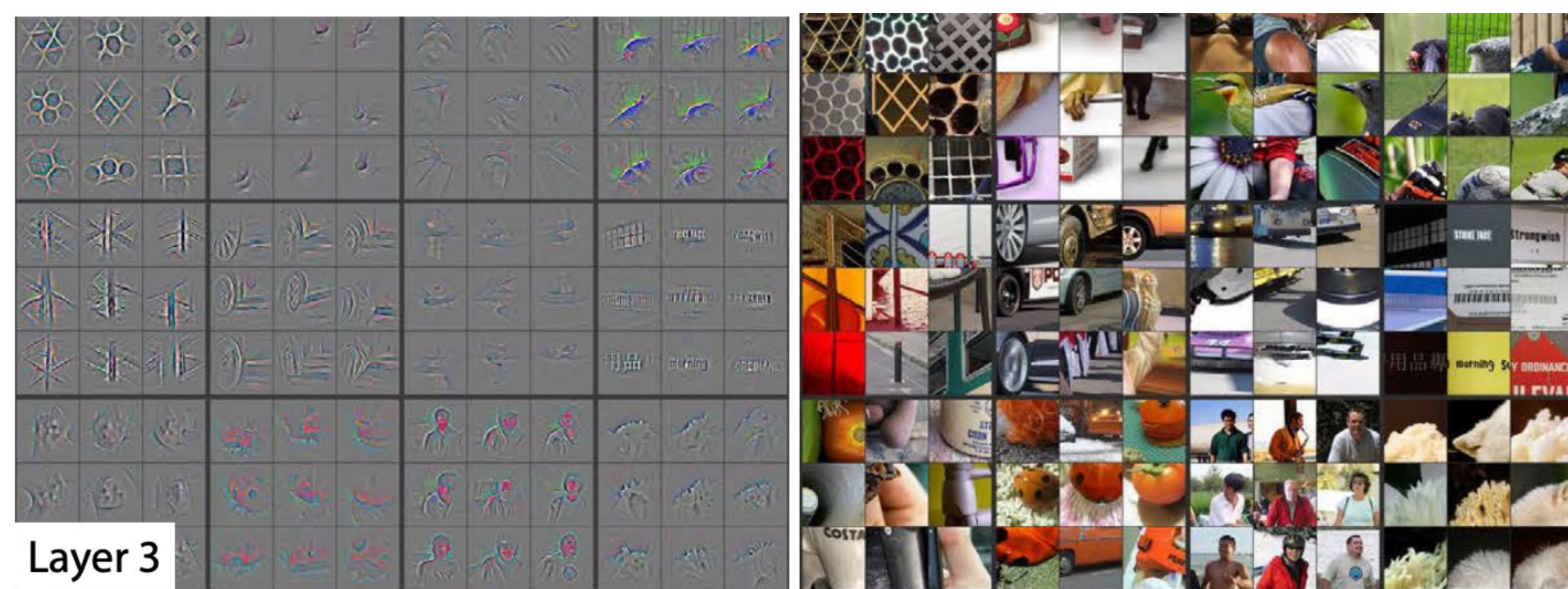


Layer 2



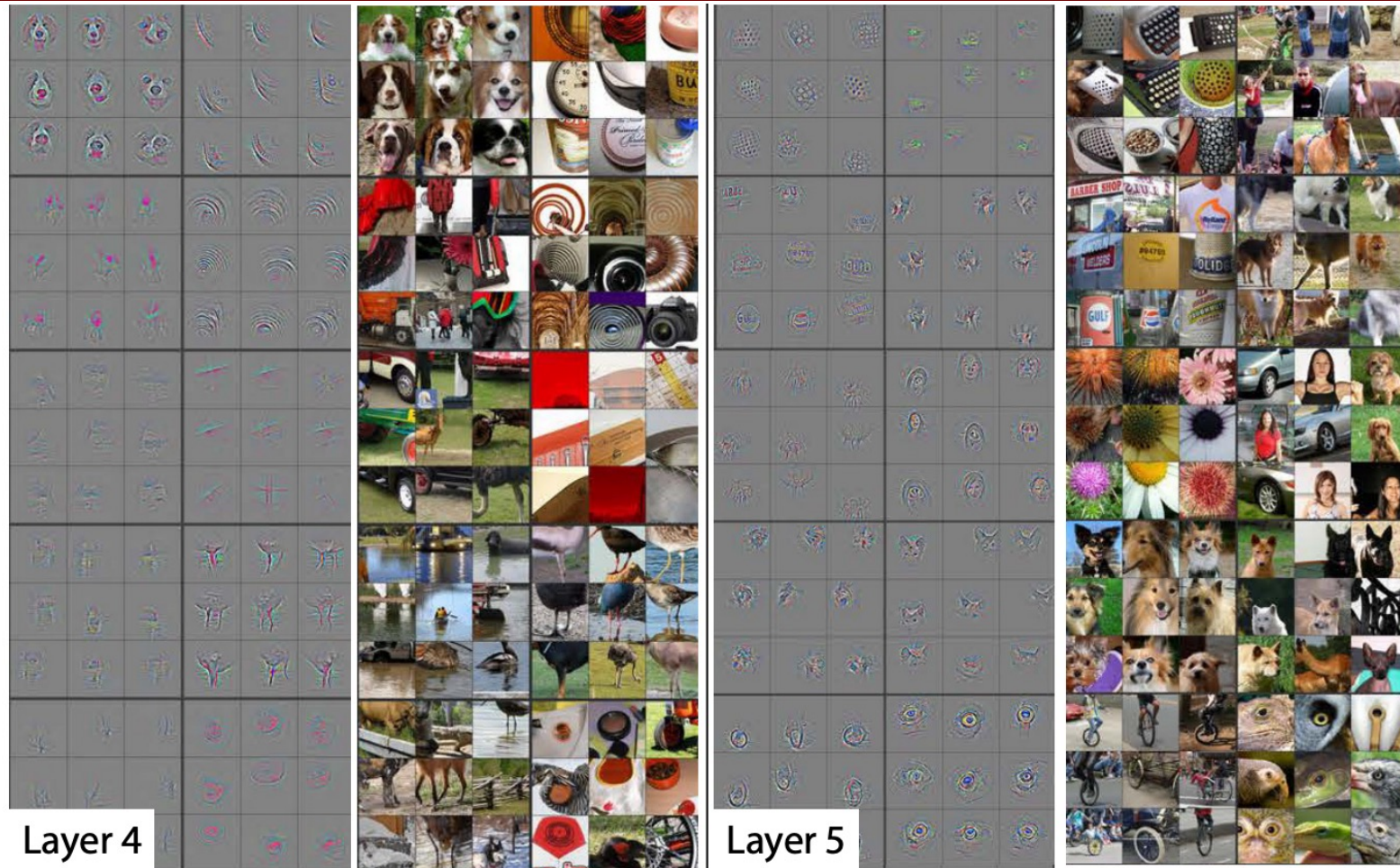
ImageNet trained network images © Matthew D. Zeiler and Rob Fergus/Springer. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use>.

A network trained on ImageNet and that does well on it has essentially developed a **smart, hierarchical representation** of these objects **across its layers**



ImageNet trained network images © Matthew D. Zeiler and Rob Fergus/Springer. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use>.

A network trained on ImageNet and that does well on it has essentially developed a **smart, hierarchical representation** of these objects **across its layers**

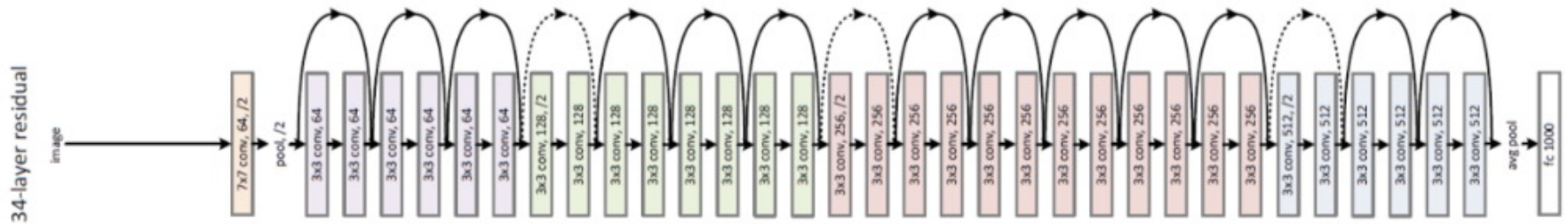


<https://cs.nyu.edu/~fergus/papers/zeilerECCV2014.pdf>

ImageNet trained network images © Matthew D. Zeiler and Rob Fergus/Springer. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use>.

The ResNet family of networks was trained on ImageNet and did very well in the associated ImageNet competition

ResNet 34*

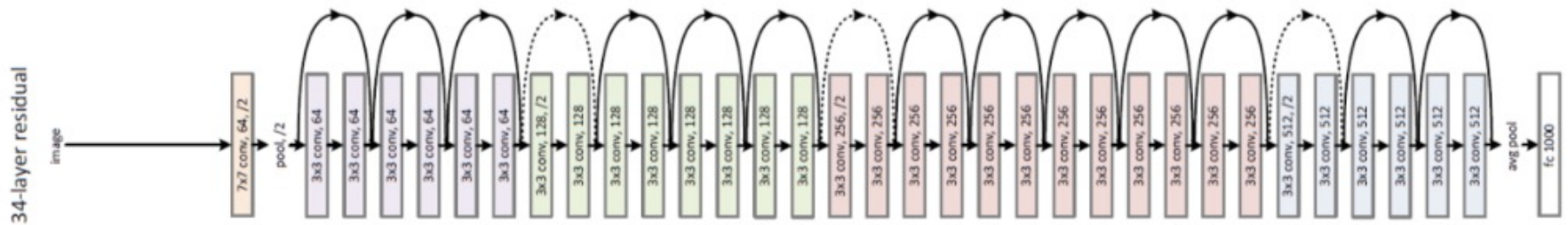


We'd expect the (learned) weights and biases of ResNet layers to embody “knowledge” (like illustrated in the previous slides) about the characteristics of the millions of everyday images that the network was trained on

ResNet34 layers image © Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun/arXiv. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use>.

But we can't use ResNet as is

ResNet 34*



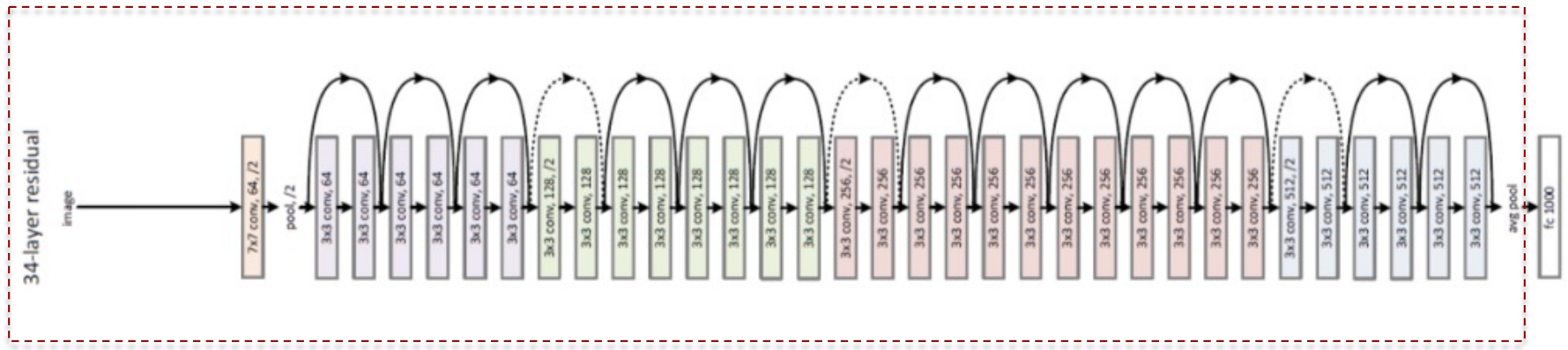
Remember that ResNet was designed to classify the image into **1000** categories. Our problem is different – we only care about **two** categories: handbags and shoes.

ResNet34 layers image © Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun/arXiv. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use>.

*<https://arxiv.org/abs/1512.03385>

So we take ResNet and stop just before the last layer

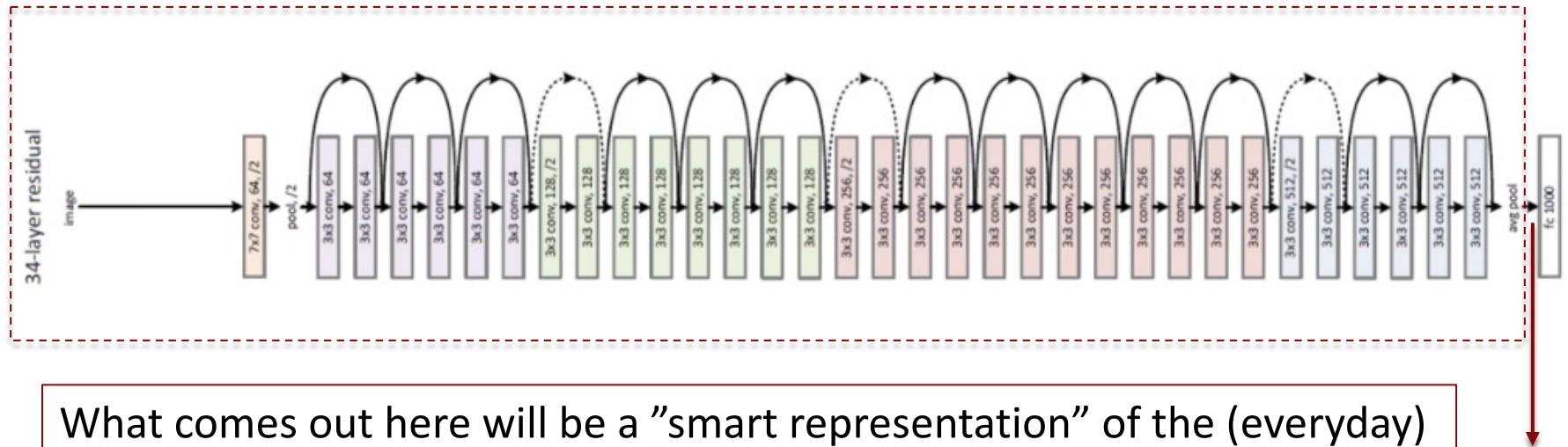
ResNet 34



ResNet34 layers image © Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun/arXiv. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use>.

We can run our images through this “headless” ResNet

ResNet 34



What comes out here will be a “smart representation” of the (everyday) image that can be used for different kinds of categorization.

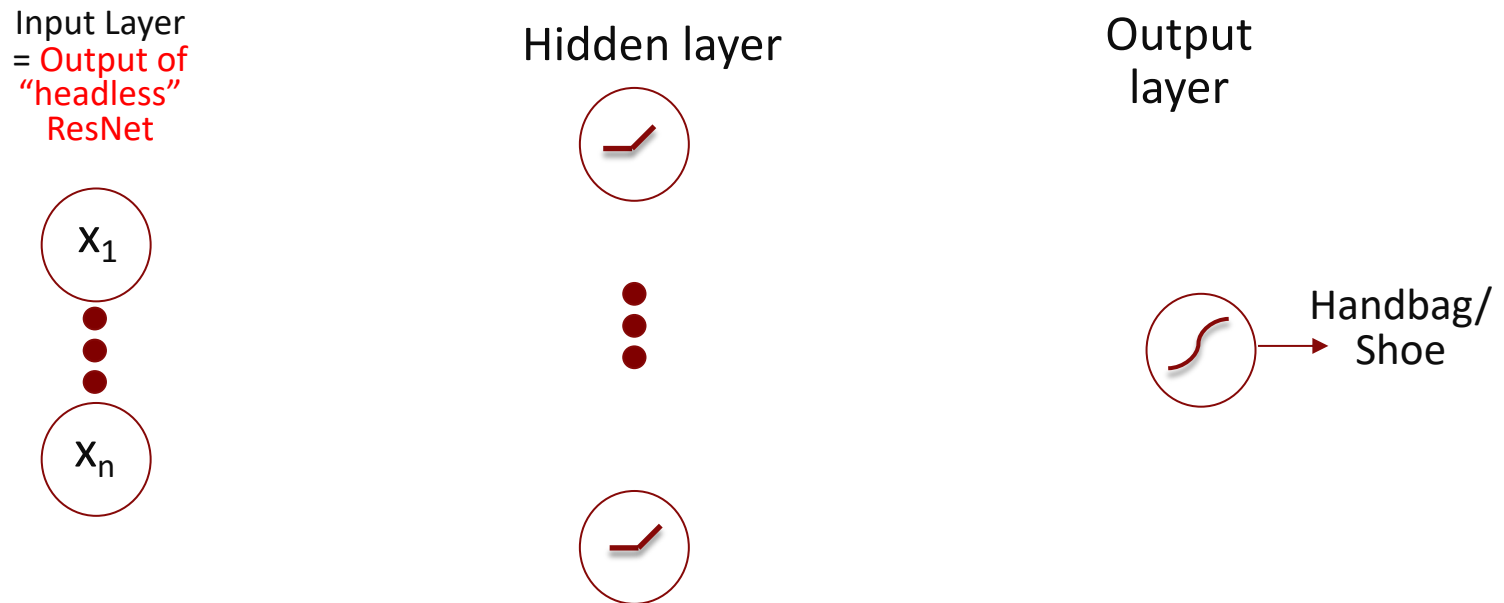
Imagine that the image gets “tagged” with lots of general tags and we can use the tags any way we wish.

*<https://arxiv.org/abs/1512.03385>

We can think of the output of “headless ResNet” as a smart representation of the raw image and use it to train a simple one-hidden-layer NN



We can think of the output of “headless ResNet” as a smart representation of the raw image and use it to train a simple one-hidden-layer NN

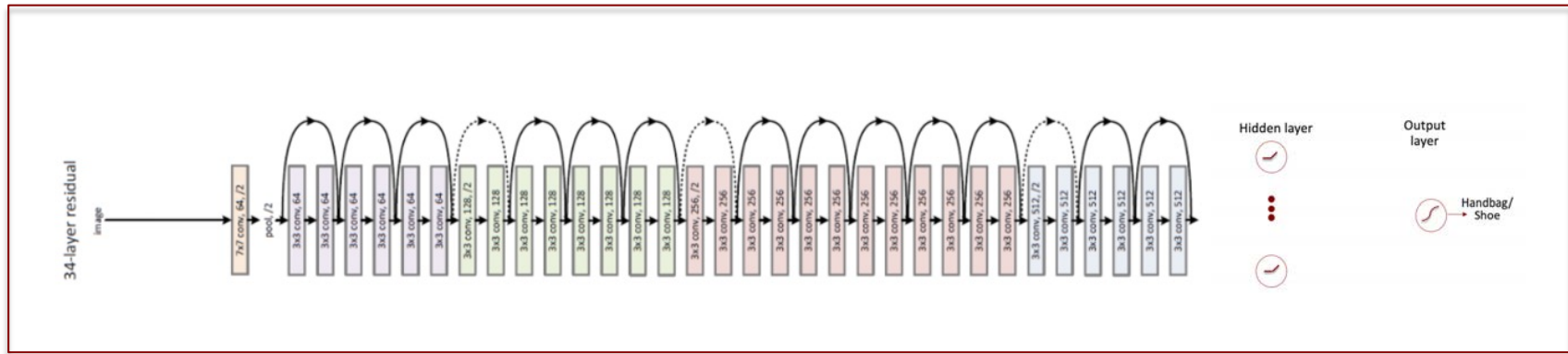


Since the inputs to the hidden layer are at a higher level of abstraction, it may be able to learn to classify handbags from shoes with very few examples

That's the basic idea behind transfer learning ...



That's the basic idea behind transfer learning **but you can get fancier**



- You can connect up “headless ResNet” with our little network and train the entire network **end-to-end**. This is called fine-tuning.
- However, you **MUST** start the training with the weights and biases that came with ResNet, rather than start from scratch.
- You will explore this in HW 1

You can find pretrained models in the TensorFlow Hub ...



TensorFlow Hub is a repository of trained machine learning models.

TensorFlow Hub is a repository of trained machine learning models ready for fine-tuning and deployable anywhere. Reuse trained models like BERT and Faster R-CNN with just a few lines of code.



[See the guide](#)

Learn about how to use TensorFlow Hub and how it works.



[See tutorials](#)

Tutorials show you end-to-end examples using TensorFlow Hub.



[See models](#)

Find trained TF, TFLite, and TF.js models for your use case.

```
!pip install --upgrade tensorflow_hub

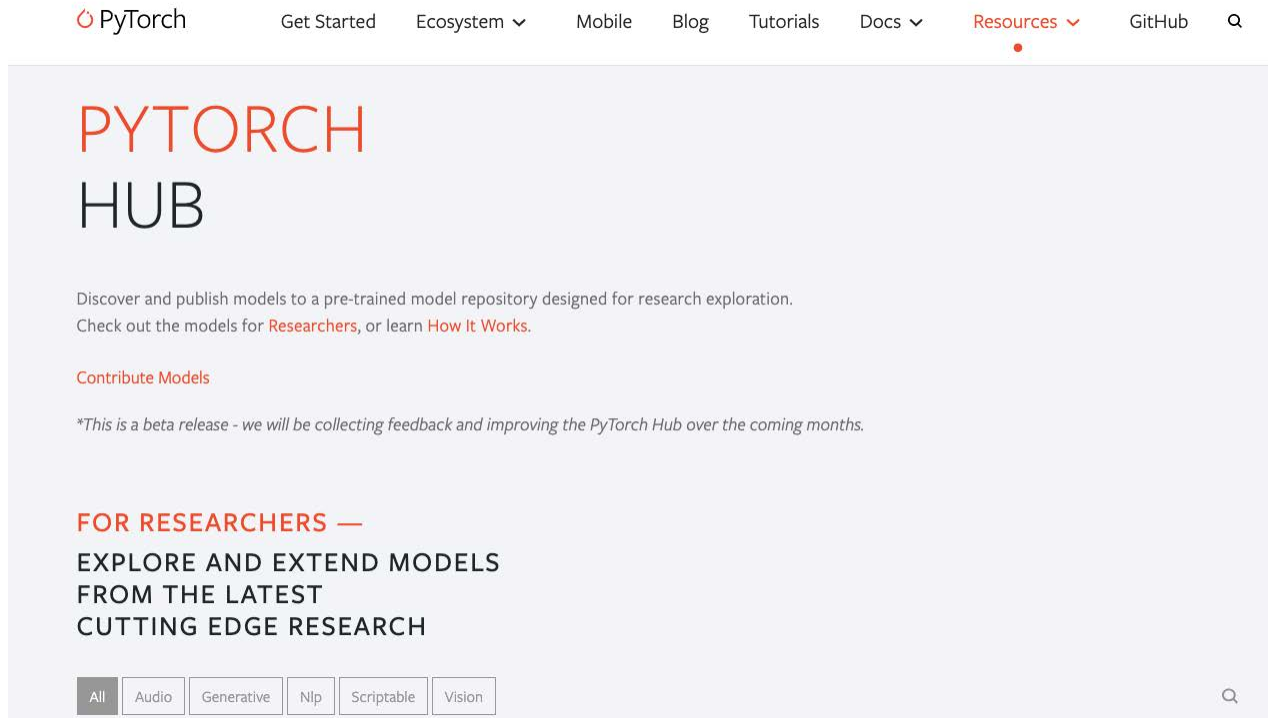
import tensorflow_hub as hub

model = hub.KerasLayer("https://tfhub.dev/google/nnlm-en-dim128/2")
embeddings = model(["The rain in Spain.", "falls",
                    "mainly", "In the plain!"])

print(embeddings.shape)  #(4,128)
```

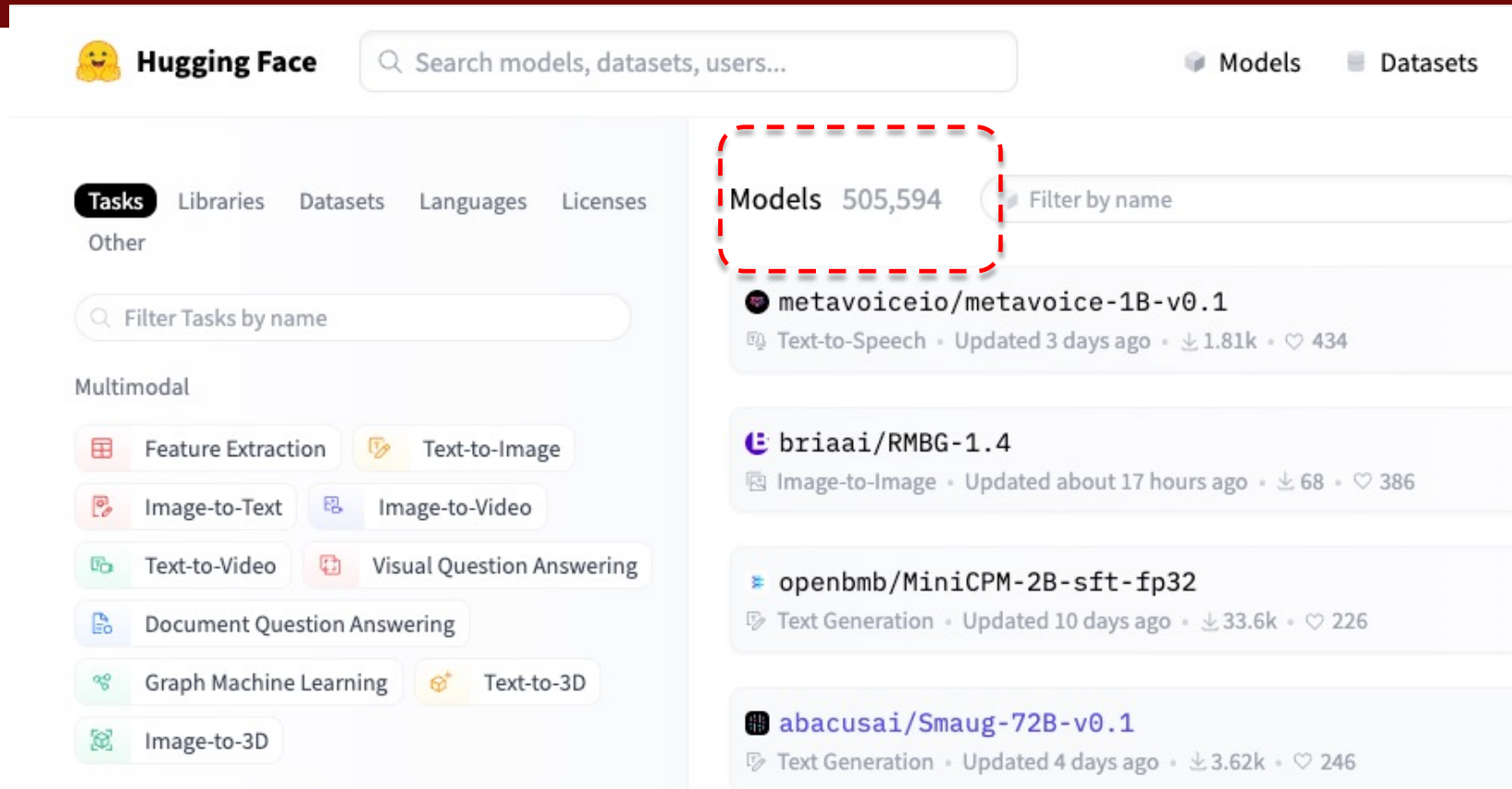
<https://www.tensorflow.org/hub>

... the PyTorch Hub, ...



<https://pytorch.org/hub/>

... and the Hugging Face Hub



The screenshot displays the Hugging Face Hub interface. At the top, the Hugging Face logo is on the left, and a search bar labeled 'Search models, datasets, users...' is in the center. To the right of the search bar are links for 'Models' and 'Datasets'. Below the search bar, there are tabs for 'Tasks', 'Libraries', 'Datasets', 'Languages', and 'Licenses'. The 'Tasks' tab is selected. Under 'Tasks', there is a search bar labeled 'Filter Tasks by name' and a section titled 'Multimodal' with various task categories like 'Feature Extraction', 'Text-to-Image', 'Image-to-Text', 'Image-to-Video', 'Text-to-Video', 'Visual Question Answering', 'Document Question Answering', 'Graph Machine Learning', and 'Image-to-3D'. On the right side, the 'Models' section is highlighted with a red dashed box, showing 'Models 505,594' and a 'Filter by name' button. Below this, several model cards are listed, including 'metavoicedio/metavoicedio-1B-v0.1', 'briaai/RMBG-1.4', 'openbmb/MiniCPM-2B-sft-fp32', and 'abacusai/Smaug-72B-v0.1'.

Hugging Face Search models, datasets, users... Models Datasets

Tasks Libraries Datasets Languages Licenses Other

Filter Tasks by name

Multimodal

- Feature Extraction Text-to-Image
- Image-to-Text Image-to-Video
- Text-to-Video Visual Question Answering
- Document Question Answering
- Graph Machine Learning Text-to-3D
- Image-to-3D

Models 505,594 Filter by name

- metavoicedio/metavoicedio-1B-v0.1**
Text-to-Speech • Updated 3 days ago • 1.81k • 434
- briaai/RMBG-1.4**
Image-to-Image • Updated about 17 hours ago • 68 • 386
- openbmb/MiniCPM-2B-sft-fp32**
Text Generation • Updated 10 days ago • 33.6k • 226
- abacusai/Smaug-72B-v0.1**
Text Generation • Updated 4 days ago • 3.62k • 246

Back to Colab

[Link to colab](#)

Appendix

The convolutional filter is just a slightly modified neuron

- A “traditional” neuron (in the first hidden layer) is connected to all pixels of the input image. In contrast, a “convolutional filter” neuron is connected only to pixels in a small *region* of the input image
- Sliding the filter across the image $\Leftrightarrow$ can be thought of as a different filter for each window *but with the same weights*
- Benefits
 - Preserves local adjacency
 - Far fewer parameters
 - Translation invariance – can detect the same pattern regardless of where it appears in an image

MIT OpenCourseWare
<https://ocw.mit.edu>

15.773 Hands-on Deep Learning

Spring 2024

For information about citing these materials or our Terms of Use, visit: <https://ocw.mit.edu/terms>.